EXPERIMENT NO: 8

Student Name: KJ Pritpal Singh
UID: 23BCS14164
Branch: BE-CSE
Section/Group: KRG-1B
Semester: 6th
Date of Performance: 24/03/2026
Subject Name: System Design
Subject Code: 23CSH-314

Aim

To design a scalable stock trading platform similar to Zerodha/Upstox that enables real-time market data streaming, order execution, portfolio tracking, and high-frequency trading with strong consistency and low latency.

Objectives

- Understand the architecture of real-time stock trading systems.
- Identify functional requirements such as trading, portfolio tracking, and watchlists.
- Analyze non-functional requirements including latency, scalability, and consistency.
- Apply CAP theorem trade-offs in financial systems.
- Design REST and WebSocket APIs for trading workflows.

Procedure

1. Studied real-world trading platforms like Zerodha and Upstox.
2. Identified core entities such as Users, Stocks, Orders, Trades, and Portfolio.
3. Analyzed real-time stock updates using WebSockets.
4. Collected functional and non-functional requirements.
5. Designed APIs for trading, funds management, and portfolio tracking.
6. Evaluated consistency requirements for financial transactions.
7. Analyzed low-latency system design for order execution.

Functional Requirements

1. Users should be able to register, login, and complete KYC verification.
2. Users should be able to view real-time stock prices and historical data.
3. Users should be able to place, modify, and cancel buy/sell orders.
4. System should support limit and market orders with accurate status tracking.
5. Users should be able to maintain a watchlist with real-time updates.
6. System should provide a dashboard for holdings, trade history, and P&L.
7. Users should be able to deposit and withdraw funds.

Core Entities of the System

- User
- Stock

- Order
- Trade
- Portfolio
- Watchlist

API Design

1. User API

A. Signup: `POST /auth/signup`

B. KYC Verification: `POST /auth/verify-KYC`

C. Login: `POST /auth/login`

2. Market Data API

A. Live Stock Feed (WebSocket): `WS /api/v1/stocks`

B. Stock Details: `GET /api/v1/stocks/{symbol}`

C. Historical Data: `GET /api/v1/stocks/{symbol}/history`

3. Funds API

A. Deposit Funds: `POST /api/v1/funds/deposit`

B. Withdraw Funds: `POST /api/v1/funds/withdraw`

C. Transaction History: `GET /api/v1/funds/history`

4. Trading API

A. Place Order: `POST /api/v1/orders`

B. Get Orders: `GET /api/v1/users/{user_id}/orders`

C. Order Status: `GET /api/v1/orders/{order_id}`

D. Cancel Order: `DELETE /api/v1/orders/{order_id}`

5. Portfolio API

A. Portfolio Overview: `GET /api/v1/portfolio`

B. Holdings: `GET /api/v1/portfolio/holdings`

C. Positions: `GET /api/v1/portfolio/positions`

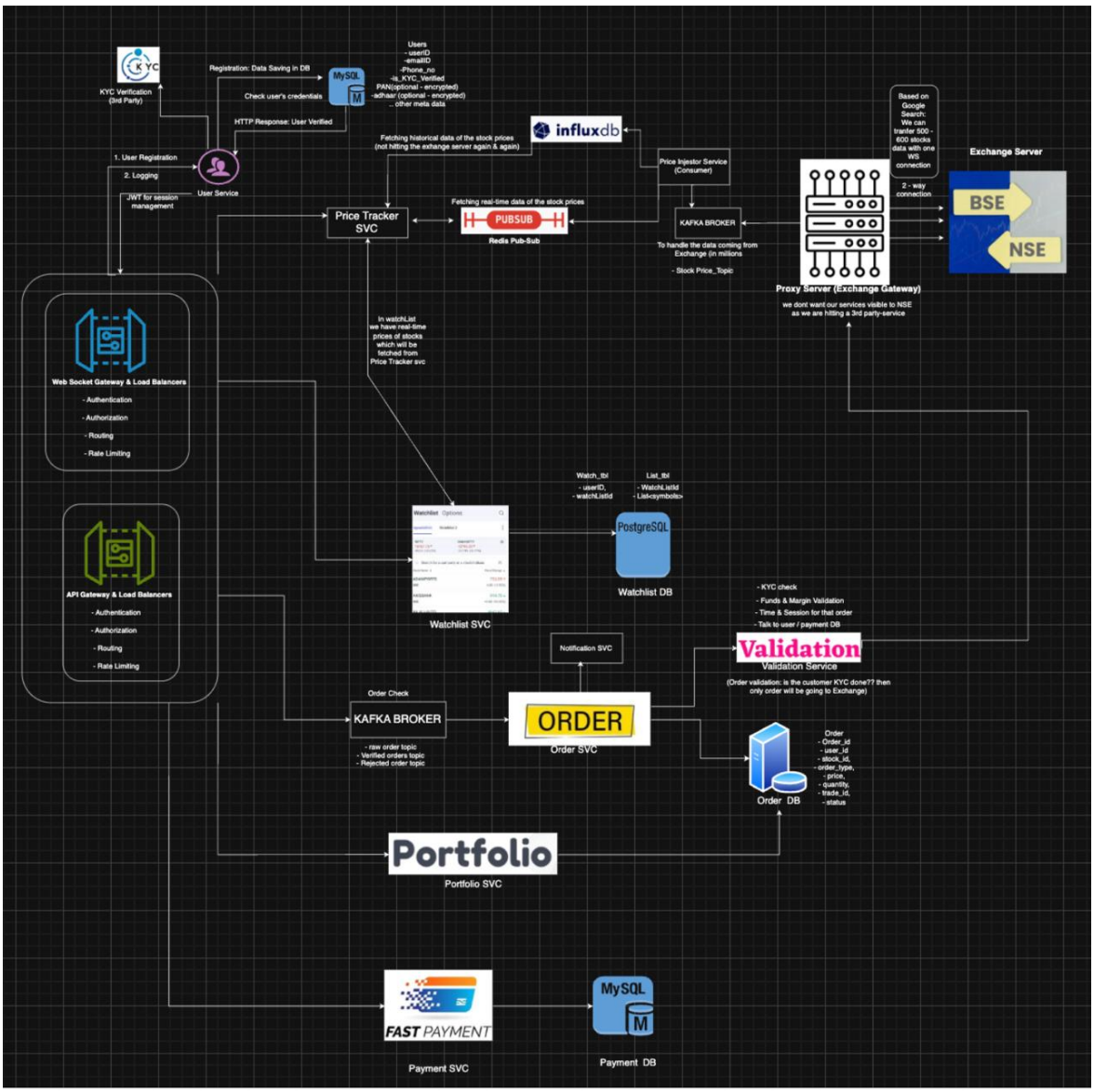
D. Performance: `GET /api/v1/portfolio/performance`

Non-Functional Requirements

1. System should support 2–3 million daily active users.
2. System should handle 8,000–10,000 stocks.
3. Strong consistency is required for order execution and balances.
4. Real-time stock updates should be highly available.
5. Order placement latency should be less than 100ms.
6. Market data latency should be less than 50ms.

High Level Design (HLD)

The system consists of client applications (web and mobile), an API Gateway, and microservices such as User Service, Market Data Service, Order Management Service, Portfolio Service, and Funds Service. Real-time stock updates are delivered using WebSockets. Orders are processed through a low-latency order matching engine. Message queues are used for asynchronous processing of trades and notifications. Strong consistency is maintained for financial transactions, while market data prioritizes availability.



Outcome

- Designed a scalable stock trading system with real-time capabilities.
- Understood the importance of consistency in financial systems.
- Implemented structured API design for trading workflows.
- Learned about low-latency architecture and real-time streaming.